

Bioinformática

Alinhamentos múltiplos e Análise filogenética

Definição de alinhamento múltiplo (AM)

Alinhamento múltiplo (AM) de sequências: procedimento de comparação de **mais do que duas sequências**, procurando séries de caracteres individuais que se encontrem na mesma ordem.

Generalização do alinhamento de 2 sequências mas com complexidade superior:

O problema em geral é NP-difícil

Aplicações do AM

Identificar **zonas conservadas de proteínas**

- prováveis zonas ativas
- zonas de mutação

Prever **função / estrutura** de proteínas

Derivação de informação **filogenética** a partir das sequências

Epidemiologia

- estudar diferenças entre estirpes da mesma espécie

Exemplo de um alinhamento múltiplo

		1000	1010	1020	1030	1040	1050	1060	1070
<i>Cynopterus</i> /1-1233	GG	TCACTAAGCTCAATCCTCATCTTCAGGGCTGTGTCTGAGGACCCCAAGGC	TCCTGGTGCAGCCAT	TAGACCCAGGGAAACCT					
<i>Macrotus</i> /1-1248	GG	TCACTAAGCTCAACAACGGTCTCCAGGGCTGTGTCTGAGGACCCCAAGGC	TCATGGTACGAGCCAT	TCAGGCCCCAGGGAAACCT					
<i>Sorex</i> /1-1179	GG	TCACTAAGCTCAACACTGGTCTCCAGGGCTGTGTCTGAGGACCCCAAGGC	TCATGGTACGAGCCG	TCAGGCCCAAAGAAACCT					
<i>Spalax</i> /1-1186	GG	TGACAAAGCTCAATGCTGGCTTGCAAGGCTGTGTCAAGGATCCCAGGC	TCCTAGTGC GTGCCG	TTGGGCTTAAAGAAACTT					
<i>Steno</i> /1-1184	GG	TCACTAAGCTCAACACCGGCCTGCAGGGCCGTGTCCGAGGACCCCAAGGC	TCCTGGTGCAGGTGG	TCAGGTCCAAGGAAACCT					
<i>Sus</i> /1-1240	GG	TCACTAAGCTCAACGCTGGCCTGCAGGGCTGTGTCTGAGGACCCCAAGGC	TCCTGGTGCAGGTGG	TCAGTCCAAGAAACCT					
<i>Equus</i> /1-1177	GG	TCGCTAAGCTCAACGCTGGCCTGCAGGGCTGTCTCCGAGGACCCCAAGGC	TCCTGGTGTGGGTTCG	TCAGGTCCAAGAGACCC					
<i>Loxodonta</i> /1-1078	GG	TTGCCAAGCTCAACGCCGACCTGCAGGGCGGTGTCTGAGGACCCCAAGAC	TCCTGGTGCAGGCCCT	TCAGGCCCAGGGGAAACCT					
<i>Procavia</i> /1-1007	GG	TTGCCAAGCTCAATGCGACCTGCAGGGCAGTGTCTGAGGACCCCAAGAC	TCCTGGTCCAAGCTCT	TGGGGCTTAGAGAAACCT					
<i>Dugong</i> /1-1046	GG	TTGCCAAGCTCAACGCTGACCTGCAGGGCCGTGTCTGAGGACCCCAAGAC	TCCTCGTGCAGGCCCT	TCAGGCCCAGAGAAACCT					
<i>Orycteropus</i> /1-1232	GG	TCGCCAAGCTTAATGCTGATCTGCAGGGCTGTGTCTGAGGACCCCAAGGC	TCCTGGTGCAGGTCT	TTAAGTCCAGAGAAACCT					
<i>Elephantulus</i> /1-981	GG	TCAACAAGCTTAATGCTGACCTGCAGGGCCGTGTCTGAGGACCCCAAGGC	TCCTCGTGCAGGATC	ATAAGACAGAGAACACT					
<i>Bradypus</i> /1-1039	GG	TCGCCAAGCTCAACACCGGCCTGCAGGGCCGTGTCTGAGGACCCCAAGGC	TCCTGGTGCAGGG	- - - - -	CCAGGAGAAACT				

Consenso: GGTCAC ...

Como avaliar as soluções ?

Funções de mérito podem ser baseadas nas usadas pelo alinhamento de pares de sequências: **heurística SP (soma de pares)**

Para cada coluna do alinhamento, acumula o mérito das diversas substituições para todos os pares de sequências;

Como penalizar espaçamentos ?

Outra possibilidade:

determinar o **consenso**

calcular nº de caracteres em cada posição que coincidam com o consenso

Algoritmos de alinhamento múltiplo

Exatos – Programação dinâmica – extensão do algoritmo para 2 sequências; só podem ser usados para nº limitado de sequências e para sequências de tamanho reduzido

Algoritmos de aproximação – métodos heurísticos que não garantem solução óptima

Progressivos – iniciam com 2 sequências e vão adicionando as restantes – e.g. algoritmos da família do Clustal

Iterativos – consideram um alinhamento inicial que vai sendo melhorado

Híbridos – podem usar outros tipos de informação (e.g. estrutura 3D)

Ferramentas

Clustal-omega

Sucessor do Clustal-W

Mais eficiente computacionalmente na construção da árvore

T-coffee:

Baseado na consistência do AM com alinhamentos de pares de sequências

Usa alinhamentos globais e locais entre as sequências para definir pesos para o alinhamento dos perfis

MAFFT

Usa conceitos de Transformadas de Fourier

Muito flexível na parametrização: pode ter diferentes “trade-offs” entre eficiência e qualidade das soluções

MUSCLE

Combina métodos progressivos e iterativos; bastante eficiente

Clustal

- Um dos programas de AM mais conhecidos com várias versões disponibilizadas ao longo dos anos (quer web-based, quer linha de comando)
- Clustal 2 ainda está disponível
 - command-line version Clustal W
 - graphical version X. Precompiled executables for Linux, Mac OS X and Windows Clustal
- Clustal ómega – versão online disponível nos servidores do EBI

Representação de alinhamentos em BioPython

- O objeto **MultipleSeqAlignment** contém objetos e funções que permitem lidar com alinhamentos (com duas ou mais sequências);
- Estes objetos são usados para guardar a estrutura dos alinhamentos e não implementa métodos para a sua criação, antes permitindo fazer o interface com ferramentas das referidas anteriormente

Para mais exemplos ver o Tutorial em:

<http://biopython.org/DIST/docs/tutorial/Tutorial.html>

Capítulo 6

Objeto MultipleSeqAlignment: exemplo

seq1 =

MHQAIFIYQIGYPLKSGYIQSIRSPEYDNW

|| |||||*||||||| ||

seq2 =

MH--IFIYQIGYALKSGYIQSIRSPEY-NW

```
from Bio.SeqRecord import SeqRecord
from Bio.Align import MultipleSeqAlignment
from Bio.Seq import Seq
```

```
seqr1 = SeqRecord(Seq(seq1),id="seq1")
seqr2 = SeqRecord(Seq(seq2),id="seq2")
alin = MultipleSeqAlignment([seqr1, seqr2])
print (alin.get_alignment_length())
print (alin[:,2])
print (alin)
```

Leitura de alinhamentos: objeto AlignIO

- O objeto AlignIO funciona para os alinhamentos de forma semelhante ao SeqIO para sequências
- Permite ler e escrever alinhamentos em diversos formatos
- Funções para leitura:
 - **Bio.AlignIO.read()**: lê um único alinhamento; retorna um objeto MultipleSeqAlignment
 - **Bio.AlignIO.parse()**: lê um conjunto de alinhamentos, retornando um iterador
 - Em ambos os casos, os parâmetros obrigatórios são um nome de ficheiro (ou handle), uma String a especificar o formato

AlignIO: exemplo cont.

```
...
>>> print ("Tam. alinhamento %i" % alignment.get_alignment_length() )
Tam. alinhamento 52
>>> for record in alignment:
...     print ("%s - %s" % (record.seq, record.id) )
>>> for record in alignment:
...     if record.dbxrefs:
...         print (record.id, record.dbxrefs )
```

Exemplos de uso do iterador sobre as sequências (objetos **SeqRecord**) no alinhamento

AlignIO: exemplo com read

Formato Stockholm, usado pelo PFAM

```
>>> from Bio import AlignIO
>>> alignment = AlignIO.read("PF05371_seed.sth", "stockholm")
>>> print (alignment)
SingleLetterAlphabet() alignment with 7 rows and 52 columns
AEPNAATNYATEAMDSLKTQAID LISQTWPVVTT...SKA COATB_BPIKE/30-81
AEPNAATNYATEAMDSLKTQAID LISQTWPVVTT...SRA Q9T0Q8_BPIKE/1-52
DGTSTATSYATEAMNSLKTQATD LIDQTWPVVTS...SKA COATB_BPI22/32-83
AEGDDP---AKAAFNSLQASATEYIGYAWAMVV...SKA COATB_BPM13/24-72
AEGDDP---AKAAFDSLQASATEYIGYAWAMVV...SKA COATB_BPZJ2/1-49
AEGDDP---AKAAFDSLQASATEYIGYAWAMVV...SKA Q9T0Q9_BPFD/1-49
FAADDATSQAKAAFDSLTAQATEMSGYAWALV...SRA COATB_BPIF1/22-73
```

AlignIO: exemplo com read

Formato FASTA

```
>>> from Bio import AlignIO
>>> alignment = AlignIO.read("PF05371_seed.faa", "fasta")
>>> print (alignment )
>>> print ("tam. alinhamento %i" % alignment.get_alignment_length() )
```

Note que apenas varia o formato ... neste caso FASTA

AlignIO: exemplo com parse

Formato Phylip

```
from Bio import AlignIO
alignments = AlignIO.parse("resampled.phy", "phylip")
for alignment in alignments:
    print (alignment)
```

Note que, neste caso, são lidos vários alinhamentos sendo retornado um iterador sobre objetos MultipleSeqAlignment

AlignIO: escrita de alinhamentos

- A função **Bio.AlignIO.write()** permite escrever alinhamentos em vários formatos
- Argumentos:
 - Lista com objetos MultipleSeqAlignment
 - Nome do ficheiro (ou handle)
 - Formato (string)

AlignIO: exemplo write

```
from Bio.Seq import Seq
from Bio.SeqRecord import SeqRecord
from Bio.Align import MultipleSeqAlignment

align1 = MultipleSeqAlignment([SeqRecord(Seq("ACTGCTAGC"), id="A"),
                               SeqRecord(Seq("ACT-CTAGC"), id="B"),
                               SeqRecord(Seq("ACTGCTAGD"), id="C"), ])
align2 = MultipleSeqAlignment([ SeqRecord(Seq("TCAGC-AG"), id="D"),
                               SeqRecord(Seq("ACAGCTAG"), id="E"),
                               SeqRecord(Seq("TCAGCTAG"), id="F"), ])

my_alignments = [align1, align2]

from Bio import AlignIO
AlignIO.write(my_alignments, "my_example.phy", "phylip")
AlignIO.write(my_alignments, "my_example.sth", "stockholm")
AlignIO.write(my_alignments, "my_example.faa", "fasta")
```

AlignIO: conversão de formatos

```
from Bio import AlignIO
count = AlignIO.convert("PF05371_seed.sth", "stockholm",
                        "PF05371_seed.aln", "clustal")
print ("Convertidos %i alinhamentos" % count )
```

Equivalente a

```
from Bio import AlignIO
alignment = AlignIO.read("PF05371_seed.sth", "stockholm")
AlignIO.write([alignment], "PF05371_seed.aln", "clustal")
```

Slicing de alinhamentos: exemplos

```
alignment[3:7]
```

Dará o alinhamento com as sequências entre a 4ª e a 7ª (todas as colunas)

```
alignment[2].seq[6]
```

Uma única letra: 3ª sequência, 7ª caracter da sequência do alinhamento

```
alignment[:, 6]
```

Apenas uma coluna (a 7ª); todas as sequências

```
alignment[3:6, :6]
```

Sequências da 4ª à 6ª, primeiras 6 colunas

Módulo Pairwise

Módulo do BioPython que permite executar algoritmos de Programação Dinâmica para obter alinhamentos ótimos de pares de sequências

Permite alinhar sequências de DNA/RNA e proteínas

Permite considerar scores com valores para match/ mismatch, bem como usar matrizes de substituição existentes

Permite definir penalizações de gaps de acordo com o modelo affine gap penalty (abertura e extensão)

Ver exemplos no Notebook !!

Diamond

Diamond – breve introdução

- DIAMOND é um alinhador de sequências de proteínas e ADN traduzido, desenvolvido para ter altas "performances". As principais características são:
 - Alinhamento de sequências a uma velocidade de 100x-10,000x superior ao BLAST.
 - Alinhamentos de "frameshift" para análise de leitura longa.
 - Pouco exigente ao nível de recursos computacionais e adequado para funcionar em portáteis.
 - Vários formatos de output, incluindo BLAST pairwise, tabular e XML, bem como classificação taxonómica.

Diamond

- Parâmetros passados usados dentro do container:
 - identificação do programa: blastp, blastn, blastx and tblastx
 - **--db** identificação da Base de Dados
 - **--query** localização do ficheiro de *input*
 - **--out** especificação do ficheiro de *output*
- Exemplo :

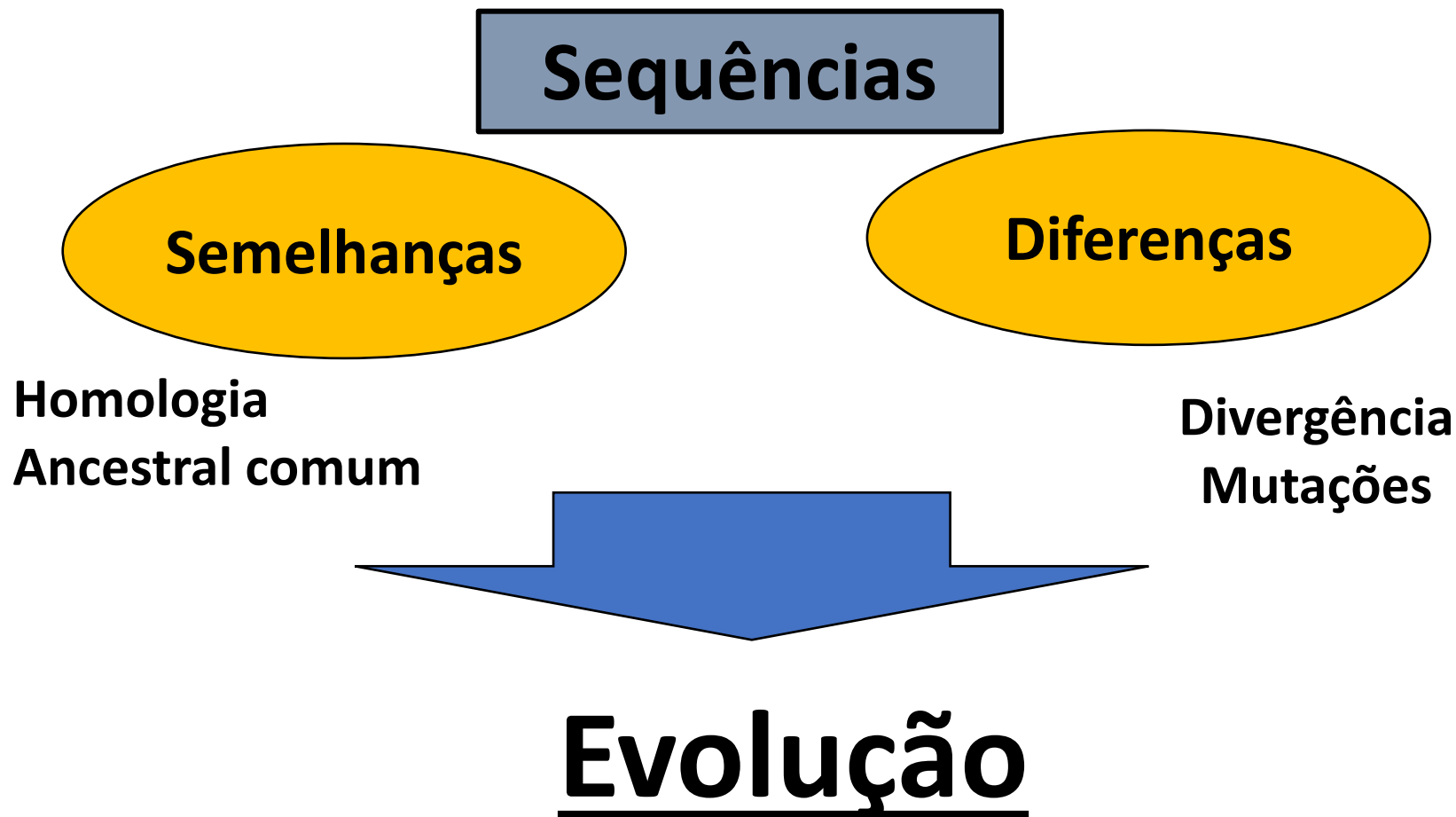
```
$ diamond blastp --db swissprot --query mypeptides.tfa --out mypeptides.out
```

Análise filogenética - definição

Análise filogenética de um conjunto de sequências (DNA, RNA, proteínas) é a determinação de como cada sequência pode ter sido derivada ao longo do processo de **evolução** natural.

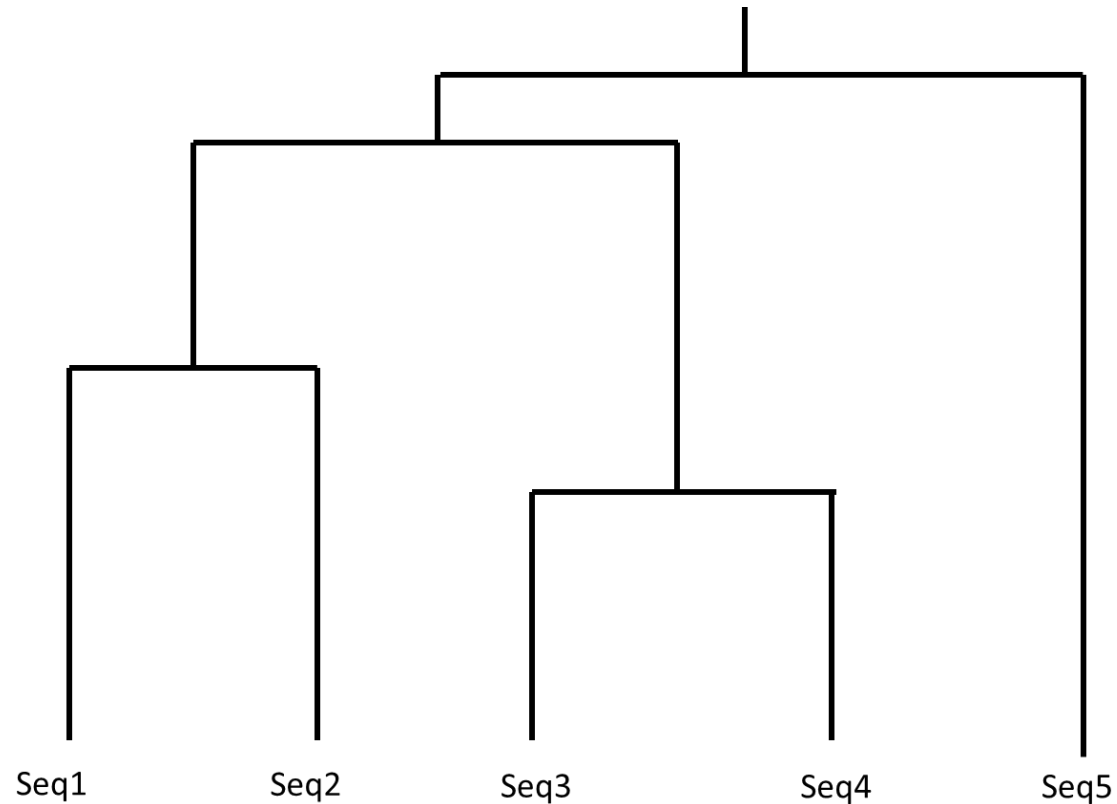
Relações evolutivas são visualizadas colocando as sequências como folhas de uma **árvore evolucionária**, onde os nós de ramificação representam eventos de mutação (substituição, inserção, remoção).

Análise filogenética



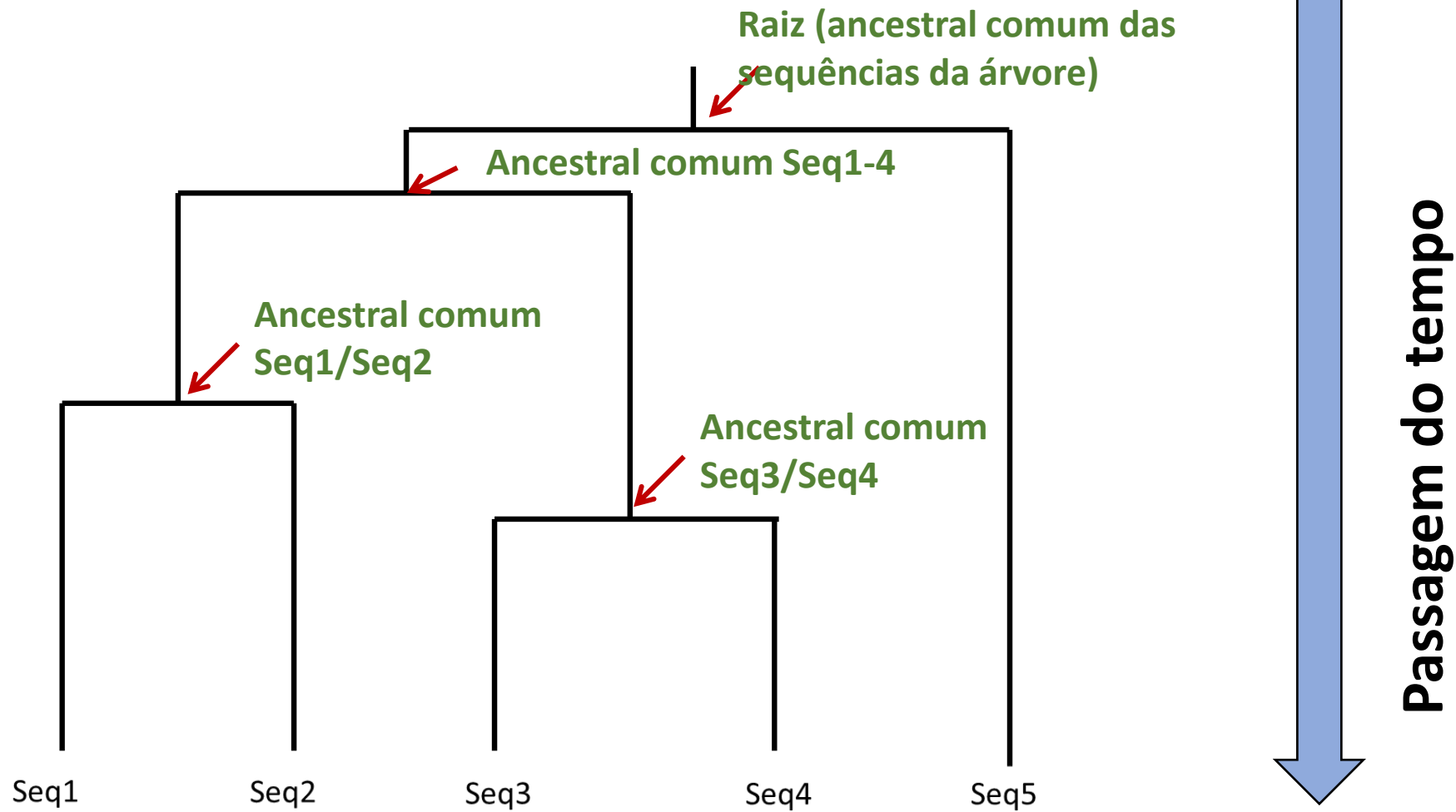
Análise filogenética

Uma árvore filogenética sugere as relações de proximidade entre sequências;
Proximidade na árvore sugere proximidade evolutiva



Análise filogenética

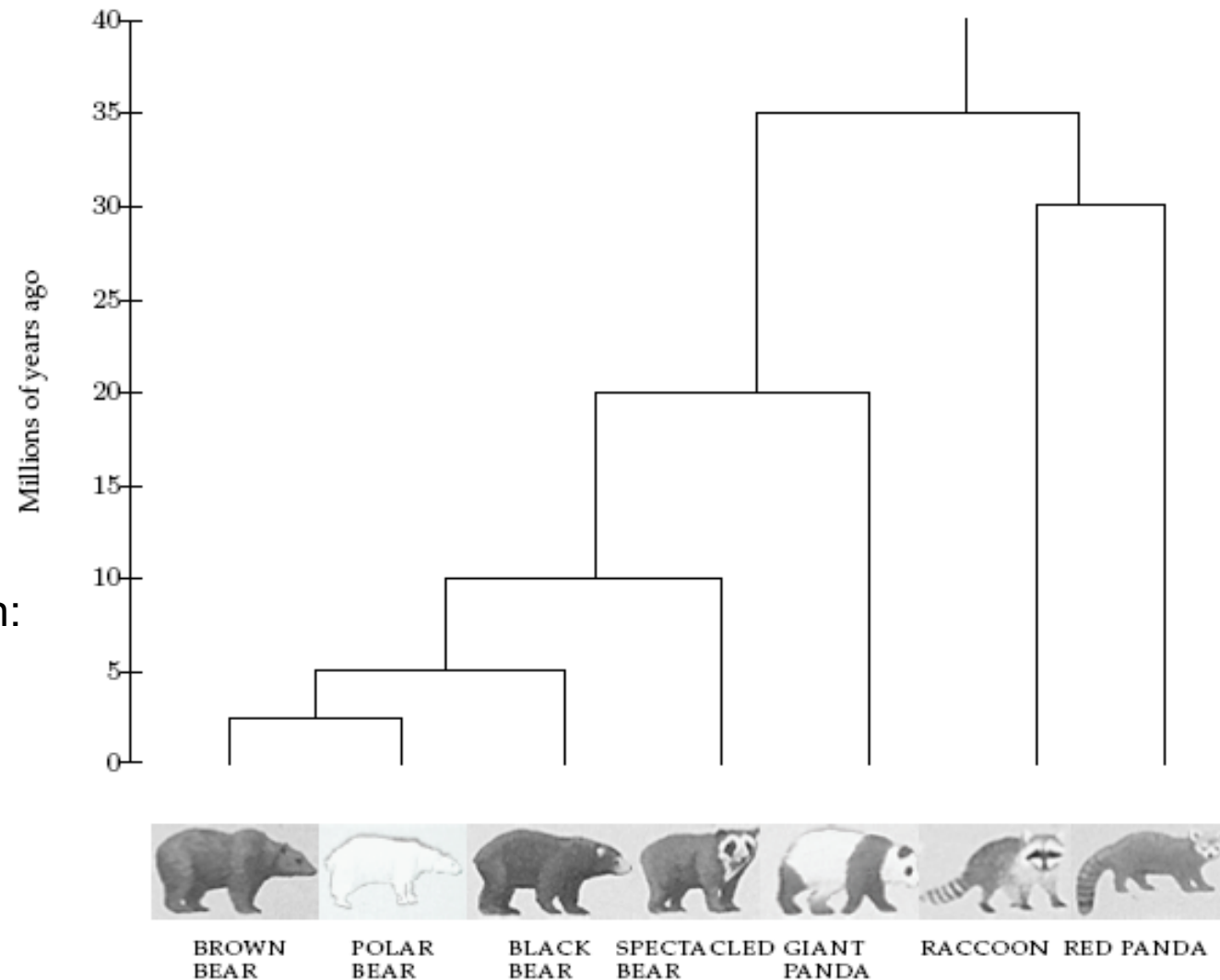
Os nós da árvore indicam ancestrais comuns;



Análise filogenética

Pode
representar
relações entre
espécies ou
outros taxa

1985, Steven O'Brien:
Identifica família do
panda usando seqs
de DNA



Análise filogenética: aplicações

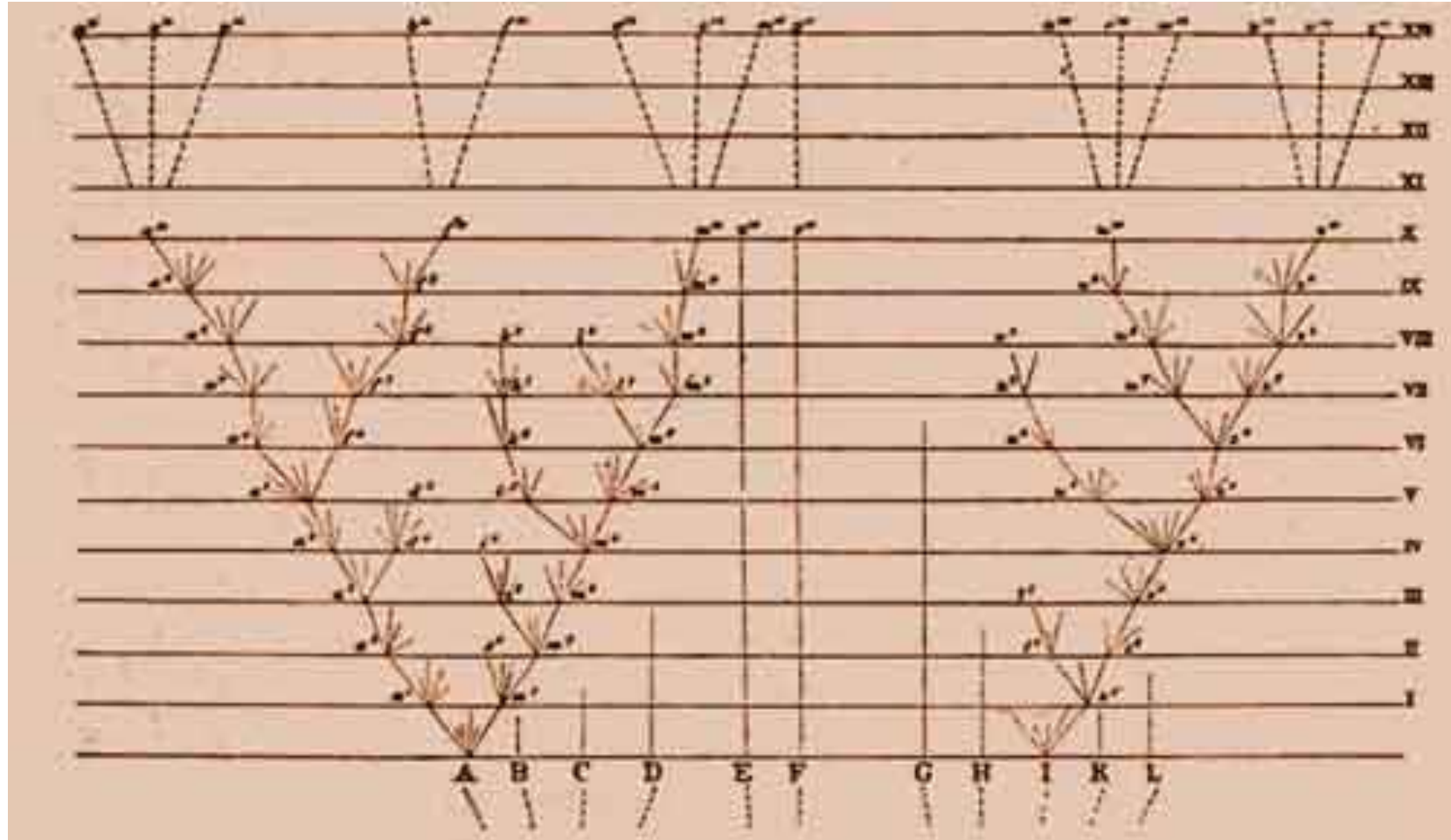
Determinar a **árvore da vida** - evolução das diferentes espécies - complementando os métodos tradicionais baseados na morfologia; permitem estabelecer relações taxonómicas entre espécies ou ancestralidade entre indivíduos ou populações;

Ajuda na determinação da **função** de sequências de DNA/ proteínas - determinação de ramos com domínios específicos que podem ter consequências funcionais;

Análise de espécies com mutações rápidas (e.g. virus) – pode ajudar na **epidemiologia**; permite hierarquizar mutações numa árvore - antigas vs. recentes;

Primeiro passo para alguns algoritmos de Alinhamento Múltiplo (progressivos).

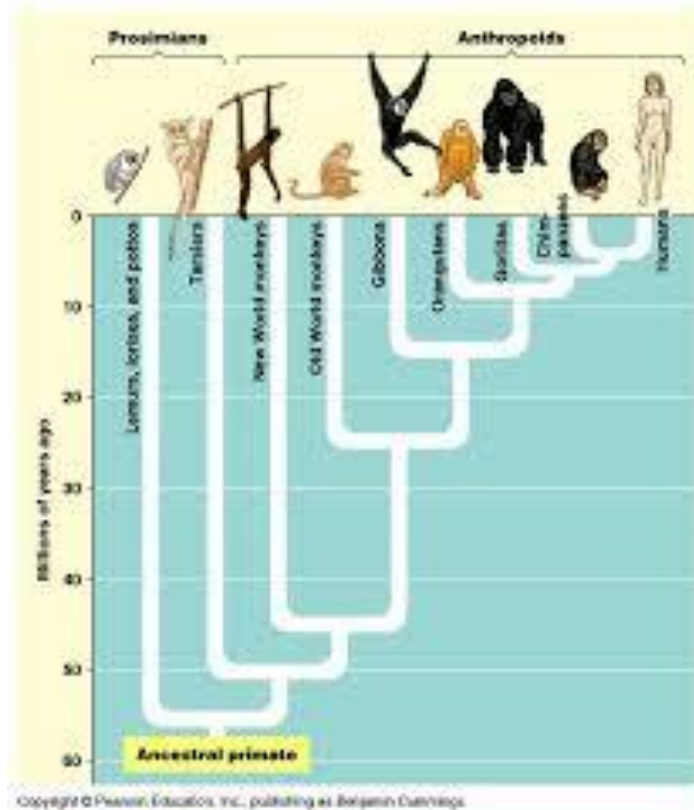
Análise filogenética



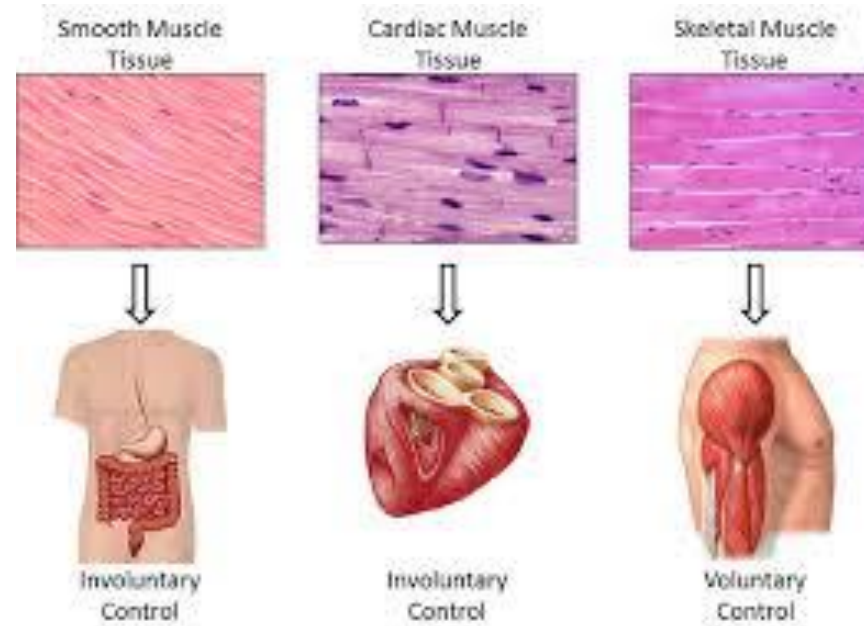
Única figura no livro “Origem das Espécies” de Darwin

Tipos de filogenias

Interespecíficas



Intra-específicas



**Tecidos
(mutações somáticas)**

Árvore de gene/sequência vs. espécie

- A evolução de um gene na maioria dos casos segue a evolução observada da espécie
- Por exemplo, a reconstrução filogenética de um gene humano terá preponderância a agrupar o gene humano com o chimpanzé, e ambos com o gorila...

Há exceções:

- Nem sempre a filogenia estará correcta; não se pode fazer inferência a partir de um só gene.
- A relação entre espécies (taxonomia) pode estar incorrecta
- Transferência horizontal

Típico em bactérias

Gene é incorporado no genoma de uma fonte exterior

Não seguiu a história evolutiva da espécie onde se inseriu

Árvores evolucionárias

Indicam o sentido da passagem do tempo

Pode assumir-se a hipótese do relógio molecular – taxas de mutação uniformes

Árvores podem ser representadas pelos clusters que se obtêm juntando taxa (folhas) presentes abaixo de cada nó interno (sub-árvores)

Nº de árvores aumenta muito rapidamente com o aumento do nº de sequências.

Algoritmos de análise filogenética

Objectivo: a partir de um **conjunto de sequências** (DNA ou proteínas), determinar a **árvore evolucionária** que melhor explique a sua evolução.

Problema de **otimização**: de entre todas as árvores possíveis, escolher a que maximiza uma dada função objetivo.

Espaço de procura tipicamente bastante grande – problema muito complexo.

Algoritmos de previsão filogenética

Baseados na **distância**

Baseia-se na distância (alterações) entre pares de sequências

Neighbor Joining, UPGMA

Máxima parcimônia (ou mínima evolução)

Retornam a árvore que minimiza nº de mutações necessárias para explicar a variação das sequências

Máxima verosimilhança

Emprega modelos probabilísticos

Métodos baseados na distância

Baseiam-se na **distância** (inverso da similaridade) entre os diversos **pares de sequências** considerados.

Objectivo: tentar identificar sequências a colocar como **vizinhas** e determinar **comprimentos dos ramos** da árvore filogenética que representem, o mais fielmente possível, as distâncias entre os pares de sequências.

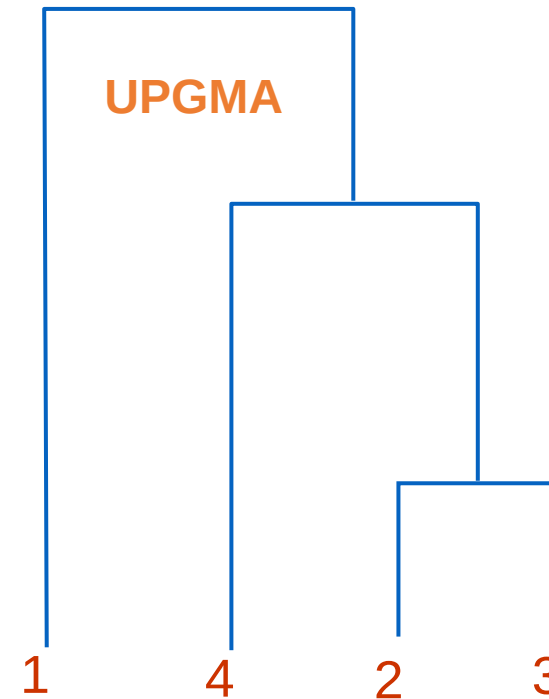
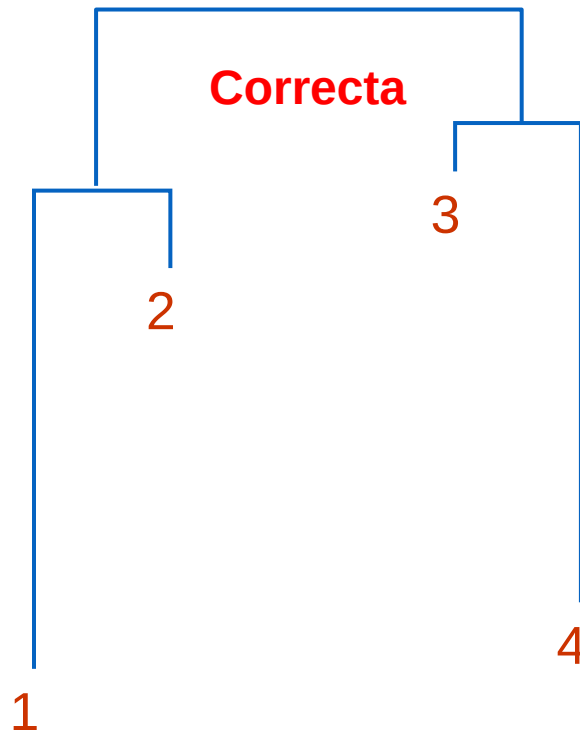
São usados como primeiro passo dos **métodos progressivos de AM** (e.g. ClustalW).

Algoritmo UPGMA

Assume taxas de mutação uniformes em todos os ramos, logo árvores criadas são **ultramétricas**, i.e., distância de todas as folhas até à raiz é a mesma.

Esta é uma das grandes limitações deste método...

E
X
E
M
P
L
O



Neighbor Joining

Funciona de forma semelhante ao UPGMA

Muda a forma de escolher os clusters que se juntam em cada passo: usa-se a regra de juntar sub-árvores que estejam próximas, mas afastadas das restantes

Muda o valor colocado como altura da árvore

Máxima parcimónia (mínima evolução)

Objectivo: **minimizar nº de passos evolutivos** (mutações) que explicam a variação das sequências

Baseia-se num **Alinhamento Múltiplo**; análise de cada posição (coluna do alinhamento)

São identificadas as árvores que requerem o menor nº de mutações, para todas as posições (informativas)

Vantagem de ser fácil estabelecer relação entre ramos da árvores e as mutações que ocorrem

Método **exato - pesado** computacionalmente – ideal para **filogenias menos profundas** mas não ideal para espécies distantes

Métodos de máxima verosimilhança

Calculam probabilidades de cada árvore dado um AM.

Incorporam um dado **modelo de evolução**.

Conseguem lidar com uma grande gama de situações dados os diversos modelos adotados

Probabilidade de uma árvore é a multiplicação das probabilidades em cada posição.

Método computacionalmente mais pesado, mas muito usados para processos evolutivos mais complexos

Aferir significância

Bootstrap

Selecionar colunas do AM aleatoriamente, com substituição;

Repetir construção da árvore com diferentes seleções;

Frequência com que dada característica ocorre é indicador da sua confiança.

Se possível usar dois tipos de métodos para construir a árvore e comparar resultados !

Programas de análise filogenética

MEGA - <http://www.megasoftware.net/>

Inclui métodos de máxima parcimónia, distância, máxima verossimilhança

Biopython

Módulos para análise filogenética

Representação de árvores

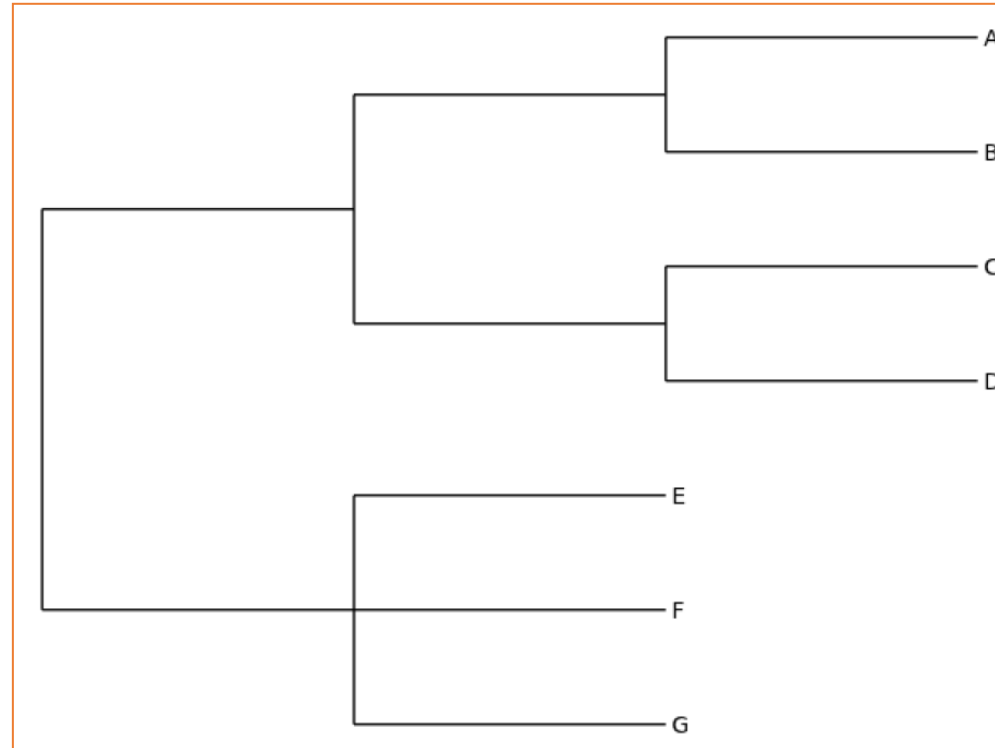
Formato Newick

```
(((A,B),(C,D)),(E,F,G))
```



Ficheiro "simple.dnd"

Representação
gráfica

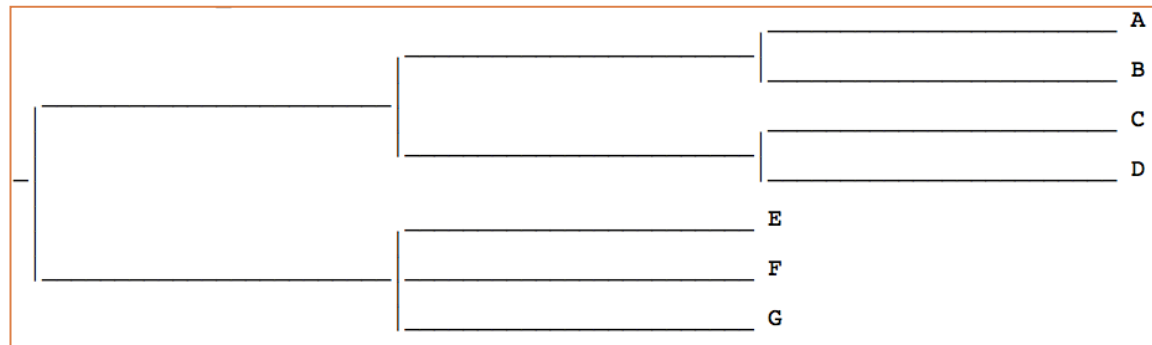


Criar árvore e desenhar

```
>>> from Bio import Phylo
>>> tree = Phylo.read("simple.dnd", "newick")
>>> print (tree)
```

```
Tree(weight=1.0, rooted=False, name="")
  Clade(branch_length=1.0)
    Clade(branch_length=1.0)
      Clade(branch_length=1.0)
        Clade(branch_length=1.0, name="A")
        Clade(branch_length=1.0, name="B")
      Clade(branch_length=1.0)
        Clade(branch_length=1.0, name="C")
        Clade(branch_length=1.0, name="D")
    Clade(branch_length=1.0)
      Clade(branch_length=1.0, name="E")
      Clade(branch_length=1.0, name="F")
      Clade(branch_length=1.0, name="G")
```

```
>>> Phylo.draw_ascii(tree)
```



Objetos Tree e Clade

- Os objetos Tree e Clade permitem representar a estrutura de uma árvore de forma recursiva
 - Tree – informação global da árvore (e.g. se tem raiz)
 - Clade – informação específica de cada nó/ramo, tal como comprimento + informação sobre as suas sub-árvores
- Existem diversos métodos para percorrer e modificar árvores e sub-árvores (ver secção 13.4 do tutorial)

Funções de I/O para árvores

- Tal como no caso do SeqIO e do AlignIO, o biopython disponibiliza com o Phylo um conjunto de funções para ler e escrever árvores em diversos formatos: parse, read, write e convert.
- Os significados de cada uma destas funções são muito semelhantes aos anteriores como se constata nos exemplos seguintes

Funções de I/O para árvores: exemplos

```
>>> from Bio import Phylo
>>> tree = Phylo.read("int_node_labels.nwk", "newick")
>>> print (tree)
>>> Phylo.draw_ascii(tree)

>>> Phylo.convert("int_node_labels.nwk", "newick", "tree.xml",
"phyloxml")

>>> trees = Phylo.parse("tree.xml", "phyloxml")
>>> for tree in trees: print(tree)
```